Implementación de un laboratorio virtual para análisis de red y ciberseguridad mediante VirtualBox, Python y Snapshots

Alumno 1	*Jose Emanuel Leguizamon*
Alumno 2	*Marcelo David Kacerovsky*
Materia	Arquitectura y Sistemas Operativos
Profesor/a	*Nicolas Llaneza y Nicolás Carcaño*
Fecha de entrega	*06/11/2026*

Índice

1. **Introducción**
2. **Objetivos**
3. **Marco Teórico**
4. **Caso Práctico**
5. **Metodología Utilizada**
6. **Desarrollo de la práctica**
7. **Resultados Obtenidos**
8. **Conclusiones**
9. **Bibliografía**
10. **Anexos**

1. Introducción

La virtualización es una tecnología que permite ejecutar sistemas operativos dentro de entornos independientes conocidos como máquinas virtuales. Gracias a ella es posible crear laboratorios de prueba aislados sin modificar directamente el sistema operativo principal del equipo.

En el ámbito de la administración de sistemas y las redes informáticas, la virtualización resulta especialmente útil para desplegar servicios, probar configuraciones y experimentar con herramientas de diagnóstico de manera segura.

En este trabajo se implementó una máquina virtual utilizando VirtualBox y Ubuntu 26.04.1 LTS como sistema operativo invitado. Sobre este entorno se instalaron diversos servicios relacionados con administración y seguridad informática, entre ellos OpenSSH, Apache, UFW, Fail2Ban y Cron.

Posteriormente se desarrolló y ejecutó un script en Python capaz de analizar direcciones IP y puertos dentro de una red local autorizada. El objetivo fue observar cómo los servicios instalados impactan sobre el comportamiento de la red y verificar los resultados mediante una herramienta desarrollada por el grupo.

Para minimizar riesgos durante las pruebas, se utilizaron snapshots de VirtualBox, permitiendo conservar estados estables de la máquina virtual y recuperar configuraciones anteriores ante errores o modificaciones no deseadas.

De esta manera, la virtualización constituyó el eje principal del proyecto, funcionando como un entorno seguro para el despliegue de servicios, el desarrollo de software y la experimentación con redes.

2. Objetivo general

Implementar un laboratorio virtual utilizando VirtualBox para desplegar servicios de red y seguridad, desarrollar una herramienta de análisis de IP y puertos mediante Python y utilizar snapshots como mecanismo de recuperación de estados del sistema.

2.1. Objetivos específicos

- Comprender el funcionamiento de la virtualización.
- Configurar una máquina virtual utilizando VirtualBox.
- Implementar Ubuntu como sistema operativo invitado.
- Instalar y administrar servicios de red.
- Configurar mecanismos básicos de seguridad.
- Desarrollar un script para el análisis de IPs y puertos.
- Ejecutar el análisis dentro de un entorno controlado.
- Utilizar snapshots para preservar configuraciones funcionales.
- Documentar el proceso y los resultados obtenidos.

3. Marco Teórico

3.1. Virtualización

La virtualización permite crear representaciones virtuales de recursos físicos mediante software. Gracias a esta tecnología es posible ejecutar varios sistemas operativos en una misma computadora física, cada uno con su propia memoria, almacenamiento y configuración.

Entre sus principales ventajas se encuentran:

- Aislamiento de entornos.
- Mejor aprovechamiento de recursos.
- Facilidad para realizar pruebas.
- Seguridad frente a errores de configuración.
- Posibilidad de crear laboratorios virtuales.

3.2. VirtualBox

VirtualBox es un software de virtualización que permite crear y administrar máquinas virtuales.

En este trabajo fue utilizado para ejecutar Ubuntu dentro de una computadora anfitriona con Windows.

VirtualBox permite gestionar:

- Recursos de hardware virtual.
- Adaptadores de red.
- Almacenamiento.
- Estados guardados.
- Snapshots o instantáneas.

3.3 Servicios de red

Para construir un entorno de pruebas realista se implementaron distintos servicios dentro de Ubuntu.

- **OpenSSH**
  - Permite la administración remota de sistemas mediante conexiones seguras.
- **Apache**
  - Servidor web ampliamente utilizado.
- **UFW**
  - Firewall encargado de administrar reglas de acceso a la red.
  - **Funciones implementadas:**
    - Bloqueo de conexiones entrantes por defecto.
    - Habilitación de OpenSSH.
    - Habilitación de Apache Secure.
- **Fail2Ban**
  - Herramienta de seguridad orientada a detectar y bloquear intentos repetidos de acceso no autorizado. Para este laboratorio, se configuró una "cárcel" (jail) específica para el servicio SSH, estableciendo un límite estricto de 3 intentos fallidos de inicio de sesión. Si un host supera este límite, su IP es baneada por el lapso de un año para mitigar eficazmente los ataques automatizados (bots).
- **Cron**
  - Sistema de planificación de tareas que permite ejecutar procesos automáticamente en horarios definidos.

3.4 Análisis de puertos

Los puertos representan puntos de comunicación utilizados por aplicaciones y servicios.

Cuando un servicio se encuentra activo suele quedar asociado a uno o más puertos específicos.

Servicio	Puerto
SSH	22
HTTP	80
HTTPS	443

La detección de puertos permite identificar qué servicios se encuentran escuchando conexiones dentro de una máquina o red determinada.

3.5 Python para análisis de red

Python posee bibliotecas que permiten trabajar con conexiones TCP/IP.

Para esta práctica se desarrolló un script capaz de:

- Detectar direcciones IP activas.
- Analizar puertos específicos.
- Registrar servicios accesibles.
- Generar reportes de resultados.

La herramienta fue ejecutada exclusivamente dentro del entorno virtualizado.

3.6 Snapshots

Una snapshot es una captura del estado de una máquina virtual en un momento determinado.

Su función consiste en permitir la recuperación rápida de configuraciones anteriores.

En este proyecto fueron utilizadas antes de:

- Instalar servicios.
- Modificar configuraciones.
- Ejecutar nuevas pruebas.
- Cambiar reglas del firewall.

3.7 Arquitectura del laboratorio

El laboratorio estuvo compuesto por:

- Sistema anfitrión: Windows 11.
- Software de virtualización: VirtualBox.
- Sistema invitado: Ubuntu 26.04.1 LTS.
- Servicios implementados: OpenSSH, Apache, UFW, Fail2Ban y Cron.
- Herramienta desarrollada: script de análisis de red en Python.
- Mecanismo de recuperación: snapshots de VirtualBox.

4. Caso Práctico

El caso práctico consistió en construir un laboratorio virtual para el despliegue y análisis de servicios de red.

Se creó una máquina virtual Ubuntu utilizando VirtualBox y posteriormente se instalaron servicios relacionados con administración y seguridad:

- OpenSSH.
- Apache.
- UFW.
- Fail2Ban.
- Cron.

Una vez configurado el entorno se desarrolló un script en Python para realizar análisis de red.

El programa fue utilizado para detectar dispositivos disponibles y verificar puertos accesibles dentro de la red autorizada.

Antes de realizar cambios importantes se creó una snapshot denominada: **Snapshot #1.**

Posteriormente se realizaron modificaciones y configuraciones adicionales, generando una segunda snapshot denominada: **Snapshot #2.**

El laboratorio fue diseñado para simular un entorno de administración de sistemas dentro de una máquina virtual Ubuntu. Sobre dicho entorno se desplegaron servicios de red y seguridad, los cuales fueron analizados mediante una herramienta desarrollada en Python capaz de detectar dispositivos activos y puertos abiertos. Las snapshots fueron utilizadas como mecanismo de recuperación para preservar configuraciones funcionales durante las etapas de prueba y configuración.

5. Metodología Utilizada

La metodología aplicada se desarrolló en las siguientes etapas:

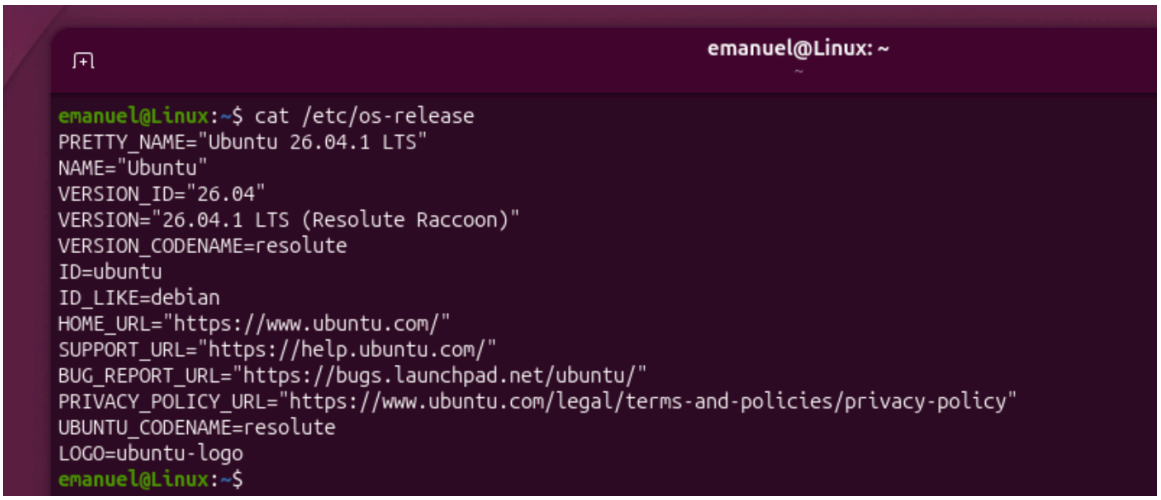
- Instalación y configuración de VirtualBox.
- Implementación de Ubuntu 26.04.1 LTS.
- Configuración de red de la máquina virtual.
- Instalación de OpenSSH.
- Instalación de Apache.
- Configuración de UFW.
- Instalación de Fail2Ban.
- Instalación de Cron.
- Creación de la snapshot inicial.
- Desarrollo del script en Python.
- Creación de una segunda snapshot.
- Ejecución del análisis de IPs y puertos.
- Muestra de los resultados.

Para la red de la máquina virtual se desactivó el modo NAT y se utilizó un Adaptador Puente (Bridged). Esto permitió que la VM obtuviera una IP en la red local y facilitó la transferencia segura del script de escaneo desde la máquina anfitriona hacia la virtual utilizando el protocolo SCP.

6. Desarrollo de la práctica

6.1 Verificación del entorno virtual

Se verificó el correcto funcionamiento de Ubuntu mediante la consulta de la versión del sistema.



6.2 Instalación de servicios

Se instalaron los siguientes componentes:

Cron

- apt install cron

Se utilizó la herramienta crontab -e para automatizar la ejecución del script de escaneo. Se configuró la siguiente tarea programada para que el análisis de red se ejecute automáticamente todos los días a las 2:30 a.m.: 30 2 * * * python3

OpenSSH

Comandos ejecutados:

- apt install ssh
- systemctl start sshd
- systemctl status sshd

```
3  apt install sshd
4  apt install ssh
5  systemctl status sshd
6  systemctl start sshd
7  systemctl status sshd
```

Fail2Ban

Configuración realizada mediante:

- nano /etc/fail2ban/jail.local

```
 8  apt install fail2ban
 9  fail2ban-client status
10  vi /etc/fail2ban/jail.local
11  nano /etc/fail2ban/jail.local
```

UFW

Configuración utilizada:

- ufw default deny incoming
- ufw default allow outgoing
- ufw allow OpenSSH
- ufw allow "Apache Secure"
- ufw enable

```
13  ufw default deny incoming
14  ufw default allow outgoing
15  ufw app list
16  ufw allow OpenSSH
17  ufw allow Apache Secure
18  ufw allow "Apache Secure"
19  ufw enable
```

6.3 Creación de Snapshots

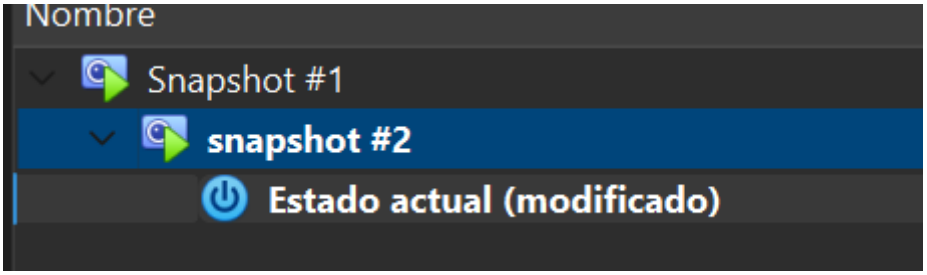
Se creó una snapshot denominada: Snapshot #1. Con la finalidad de preservar una configuración

funcional.



6.4 Modificaciones posteriores

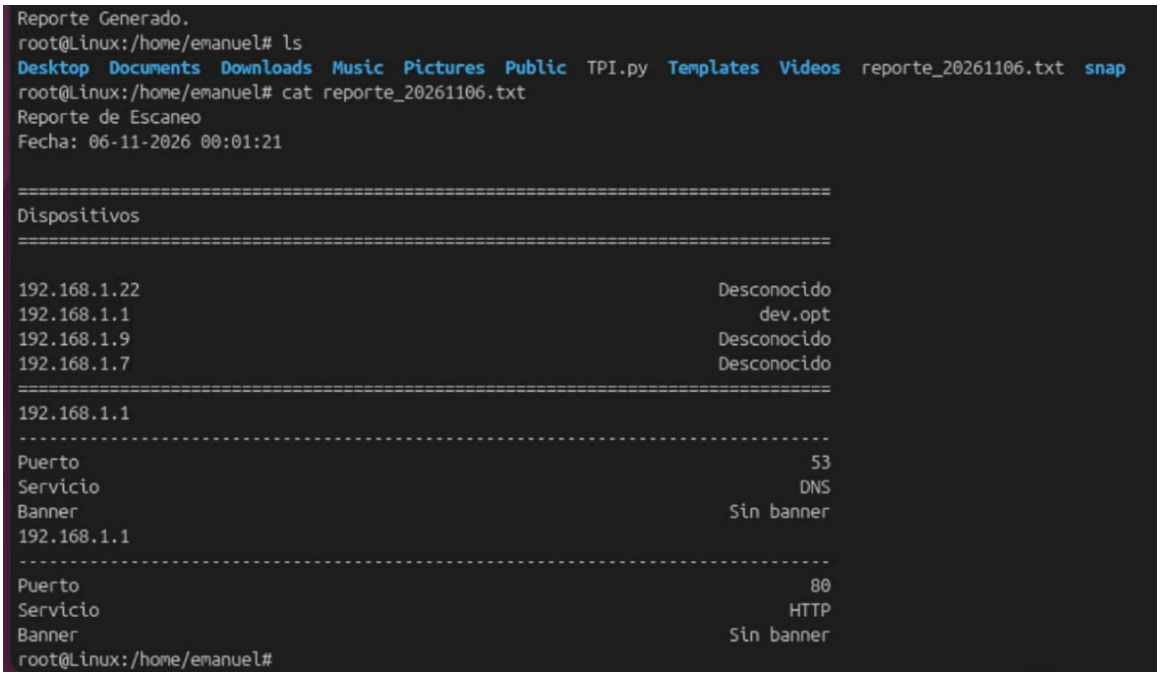
Se realizaron cambios y configuraciones adicionales dentro de Ubuntu.
Posteriormente se creó una segunda snapshot: Snapshot #2.



6.5 Ejecución del script

Se desarrolló y ejecutó un programa en Python denominado TPI.py. A diferencia de un escaneo secuencial básico, el script fue optimizado utilizando un pool de hilos (ThreadPoolExecutor) para agilizar el proceso.
La herramienta realiza las siguientes tareas:

1. Obtiene la IP local y arma un rango iterativo del 1 al 254 (excluyendo red y broadcast).
2. Envía paquetes Ping para detectar hosts activos (código de retorno 0).
3. Escanea puertos de servicios comunes (21, 22, 53, 80, 443, 8080) abriendo conexiones por sockets.
4. Captura el banner del servicio si la conexión es exitosa.
5. Exporta los resultados a un archivo de texto con marca de tiempo (ej. reporte_20261106.txt).



7. Resultados Obtenidos

La virtualización permitió construir un entorno aislado para desplegar servicios de red sin afectar al sistema operativo anfitrión.
El script desarrollado en Python fue capaz de:

- Detectar dispositivos activos.
- Verificar puertos accesibles.
- Relacionar servicios con puertos abiertos.
- Generar resultados reproducibles dentro del laboratorio.

La instalación de OpenSSH y Apache permitió observar cómo determinados servicios impactan sobre los puertos accesibles desde la red.

8. Conclusiones

La experiencia permitió comprobar que la virtualización constituye una herramienta eficaz para la construcción de laboratorios seguros de prueba y aprendizaje.

Mediante VirtualBox fue posible implementar una máquina virtual Ubuntu independiente del sistema anfitrión, sobre la cual se desplegaron servicios reales de administración y seguridad.

La ejecución del script desarrollado en Python permitió analizar direcciones IP y puertos abiertos, relacionando conceptos de programación, redes y sistemas operativos.

Por otra parte, las snapshots son un mecanismo fundamental para preservar estados funcionales y recuperar configuraciones anteriores ante posibles errores o modificaciones no deseadas.

Como propuesta de mejora futura para escalar este laboratorio, el script podría modificarse para que, en lugar de generar archivos de texto, inserte los resultados del escaneo en una base de datos relacional. Esto permitiría auditar el historial de cambios en los puertos de la red y conectar los datos a un dashboard analítico para visualizar las métricas de seguridad en tiempo real.

En conclusión, la combinación de VirtualBox, Ubuntu, servicios de red, Python y snapshots permitió desarrollar un laboratorio virtual seguro, reproducible y adecuado para tareas de administración de sistemas y análisis de redes.

9. Bibliografía

Apache Software Foundation. (2026). *Apache HTTP Server Documentation*. Recuperado de

https://httpd.apache.org/docs/

Canonical Ltd. (2026). *Ubuntu 26.04.1 LTS (Documentación oficial)*. Recuperado de https://ubuntu.com/server/docs

Fail2Ban Project. (2026). *Fail2Ban Official Documentation / Wiki*. Recuperado de https://github.com/fail2ban/fail2ban/wiki(

OpenBSD Project. (2026). *OpenSSH Manual Pages*. Recuperado de https://www.openssh.com/manual.html

Oracle Corporation. (2026). *Oracle VM VirtualBox User Manual*. Recuperado de https://www.virtualbox.org/manual/

Python Software Foundation. (2026). *Python 3 Documentation (Librerías de red y sockets)*. Recuperado de https://docs.python.org/3/

Ubuntu Community. (2026). *UFW (Uncomplicated Firewall) Documentation*. Recuperado de https://help.ubuntu.com/community/UFW

10. Anexos

Repositorio del Proyecto (Código Fuente)

Todo el código fuente desarrollado para este trabajo práctico, incluyendo el script de escaneo de red (TPI.py) debidamente documentado, se encuentra alojado en el siguiente repositorio público de GitHub:

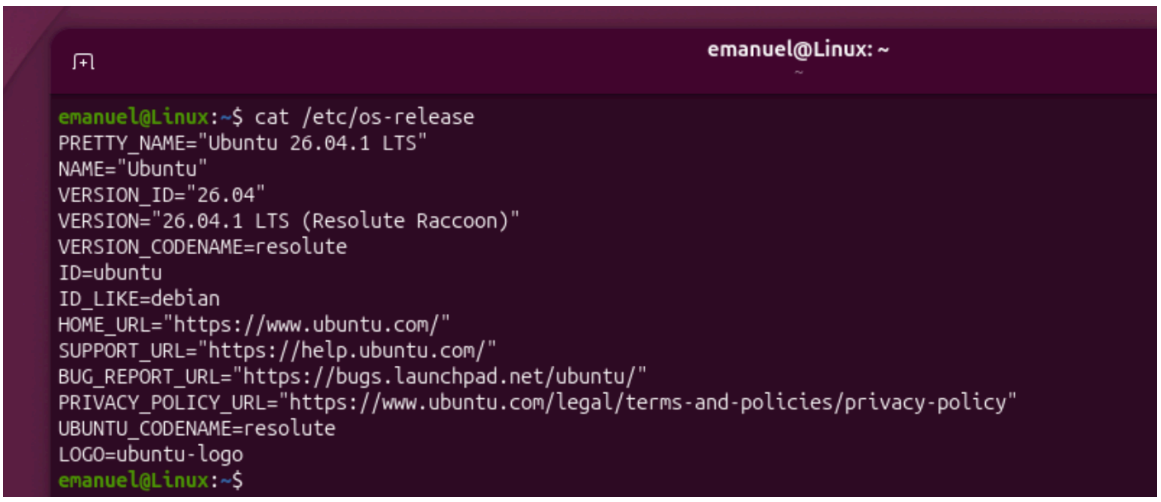
Enlace al repositorio: https://github.com/Emanuelscript/TPI-AySo-Leguizamon-y-Kacerovsky.git

Video Explicativo (Defensa Oral)

La demostración práctica del funcionamiento de la máquina virtual, la configuración de los servicios (UFW, Fail2Ban, Cron) y la ejecución del script en vivo se encuentra en el siguiente video:

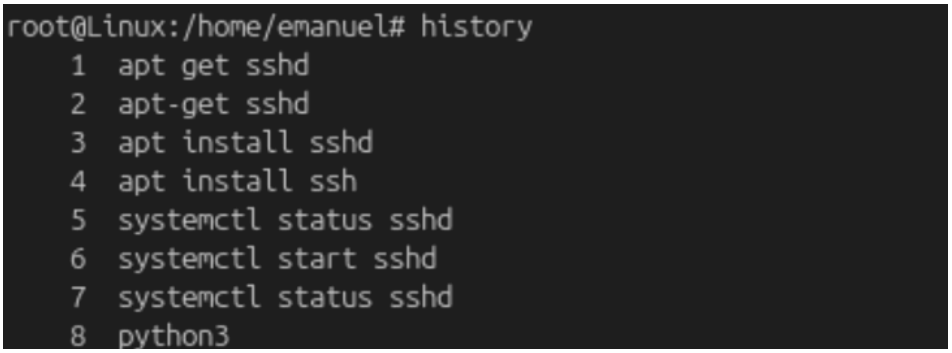
Enlace al video: https://youtu.be/T0vcR6TtXxo

Anexo A



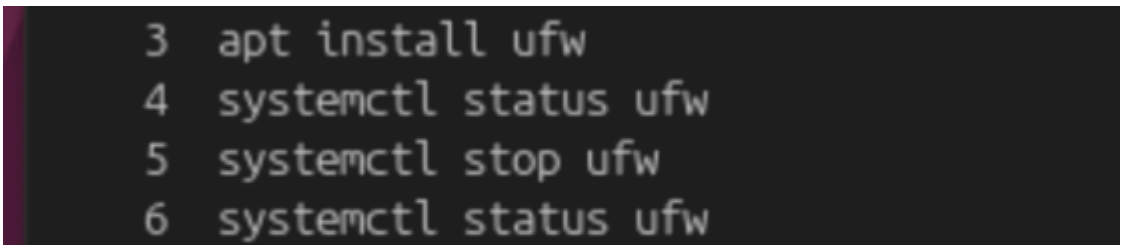
Versión de ubuntu

Anexo B



Instalación de OpenSSH.

Anexo C



Instalación de UFW.

Anexo D

```
 8  apt install fail2ban
 9  fail2ban-client status
10  vi /etc/fail2ban/jail.local
11  nano /etc/fail2ban/jail.local
```

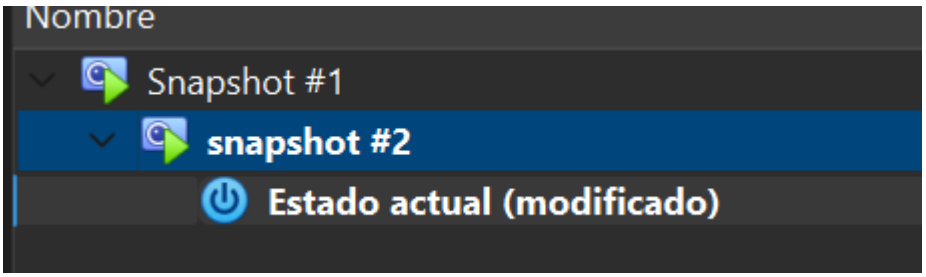
Instalación y configuración de Fail2Ban.

Anexo E



Snapshot #1.

Anexo F



Snapshot #2.